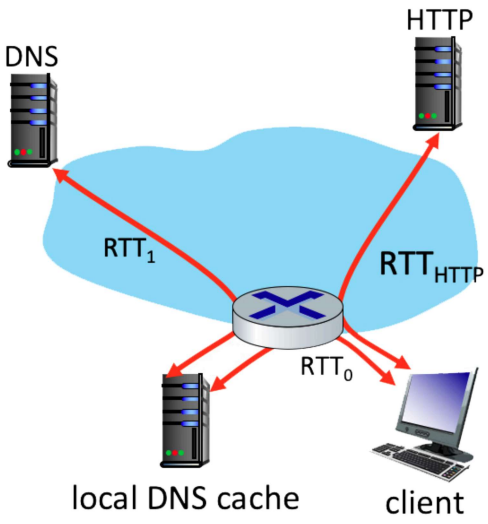
INTERACTIVE END-OF-CHAPTER EXERCISES

DNS AND HTTP DELAYS

Before doing this question, you might want to review sections 2.2.1 and 2.2.2 on HTTP (in particular the text surrounding Figure 2.7) and the operation of the DNS (in particular the text surrounding Figure 2.19).

Suppose within your Web browser you click on a link to obtain a Web page. The IP address for the associated URL is not cached in your local host, so a DNS lookup is necessary to obtain the IP address. Suppose that two DNS servers are visited before your host receives the IP address from DNS. The first DNS server visited is the local DNS cache, with an RTT delay of RTT_0 = 4 msecs. The second DNS server contacted has an RTT of 33 msecs. Initially, let's suppose that the Web page associated with the link contains exactly one object, consisting of a small amount of HTML text. Suppose the RTT between the local host and the Web server containing the object is RTT_{HTTP} = 6 msecs.



QUESTION LIST

1. Assuming zero transmission time for the HTML object, how much time (in msec) elapses from when the client clicks on the link until the client receives the object?

2. Now suppose the HTML object references 9 very small objects on the same server. Neglecting transmission times, how much time (in msec) elapses from when the client clicks on the link until the base object and all 9 additional objects are received from web server at the client, assuming non-persistent HTTP and no parallel TCP connections?

3. Suppose the HTML object references 9 very small objects on the same server, but assume that the client is configured to support a maximum of 5 parallel TCP connections, with non-persistent HTTP.

4. Suppose the HTML object references 9 very small objects on the same server, but assume that the client is configured to support a maximum of 5 parallel TCP connections, with persistent HTTP.

5. What's the fastest method we've explored: Nonpersistent-serial, Nonpersistent-parallel, or Persistent-parallel?

SOLUTION

1. The time from when the Web request is made in the browser until the page is displayed in the browser is: $RTT_0 + RTT_1 + 2*RTT_{HTTP} = 4 + 33 + 2*6 = 49$ msecs. Note that 2 RTT_{HTTP} are needed to fetch the HTML object - one RTT_{HTTP} to establish the TCP connection, and then one RTT_{HTTP} to perform the HTTP GET/response over that TCP connection.

2. The time from when the Web request is made in the browser until the page is displayed in the browser is: $RTT_0 + RTT_1 + 2*RTT_{HTTP} + 2*9*RTT_{HTTP} = 4 + 33 + 2*6 + 2*9*6 = 157$ msecs. Note that two RTT_{HTTP} delays are needed to fetch the base HTML object - one RTT_{HTTP} to establish the TCP connection, and one RTT_{HTTP} to send the HTTP request, and receive the HTTP reply. Then, serially, for *each* of the 9 embedded objects, a delay of $2*RTT_{HTTP}$ is needed - one RTT_{HTTP} to establish the TCP connection and then one RTT_{HTTP} to perform the HTTP GET/response over that TCP connection.

3. Since there are 9 objects, there's a delay of 37 msec for the DNS query, two RTT_{HTTP} for the base page, and $4*RTT_{HTTP}$ for the objects since the requests for 5 of these objects can be run in parallel ($2 RTT_{HTTP}$) and the rest can be done after ($2 RTT_{HTTP}$). The total is 37 + 12 + 12 + 12 = 73 msec. As in 2 above, $2 RTT_{HTTP}$ are needed to fetch the base HTML object - one RTT_{HTTP} to establish the TCP connection, and one RTT_{HTTP} to send the HTTP request and receive the HTTP reply containing the base HTML object. Once the base object is received at the client, the 9 HTTP GETS for the embedded objects can proceed in parallel. Each (in parallel) requires two RTT_{HTTP} delays - one RTT_{HTTP} to set up the TCP connection, and one RTT_{HTTP} to perform the HTTP GET/response for an embedded object.

4. Since there are 9 objects, there's a delay of 37 msec for the DNS query. There's also a delay of two RTT_{HTTP} for the base page, and $2 RTT_{HTTP}$ for the objects. The total is 37 + 12 + 12 = 61 msec.As in 2 and 3 above, two RTT_{HTTP} delays are needed to fetch the base HTML object - one RTT_{HTTP} to establish the TCP connection, and one RTT_{HTTP} to send the HTTP request, and receive the HTTP reply containing the base HTML object. However, with persistent HTTP, this TCP connection will remain open for future HTTP requests, which will therefore not incur a TCP establishment delay. Once the base object is received at the client, the maximum of five requests can proceed in parallel, each retrieving one of the 9 embedded objects. Each (in parallel) requires only one RTT_{HTTP} delay to perform the HTTP GET/response for an embedded object. Once these first five objects have been retrieved, (if necessary) the remaining embedded objects can be retrieved (in parallel). This second round of HTTP GET/response to retreive the remaining embedded objects takes only one more RTT_{HTTP}, since the TCP connection has remained open.

5. The delay when using persistent parallel connections is faster than using nonpersistent parallel connections, which is faster than using nonpersistent serial connections.

QUESTION 1 OF 5

Assuming zero transmission time for the HTML object, how much time (in msec) elapses from when the client clicks on the link until the client receives the object?

Answer

CHECK HINT SKIP

We gratefully acknowledge the programming and problem design work of John Broderick (UMass '21), which has really helped to substantially improve this site.

Copyright © 2010-2022 J.F. Kurose, K.W. Ross
Comments welcome and appreciated: kurose@cs.umass.edu